

FreeRTOS on PIC18F4520 MCU



NIDHI KATHURIA

FreeRTOS (pronounced *free-arr-toss*) is an open source real-time operating system (RTOS). It was designed by Richard Berry in 2002 for embedded systems. It is very popular as it is small, simple and easy to use, and supports different architectures and compiler tool chains. It has good portability and has been ported to more than 30 microcontrollers (MCUs). It is distributed under GPL with an exception, which permits users' proprietary code to remain closed source while maintaining the kernel itself as open source, thereby facilitating the use of FreeRTOS in proprietary applications.

This article deals with the basic functionality of FreeRTOS, focusing on creation and configuration of a FreeRTOS project and use of libraries.

What is a general-purpose OS

An OS is a computer program that supports a computer's basic functions and provides services to other programs (or applications) that run on the computer. The applications provide the functionality that the user of the computer wants or needs. Services provided by the OS make writing the applications faster, simpler and more maintainable.

What is an RTOS

In general, an OS is responsible for managing the hardware resources of a computer and hosting applications that run on the computer. An RTOS performs these tasks too, but is also specially designed to run applications with very precise timing and a high degree of reliability. An RTOS guarantees a certain capability within a specified time constraint. The main difference is the time constraint related to the actions that are being performed. In a real-time application,

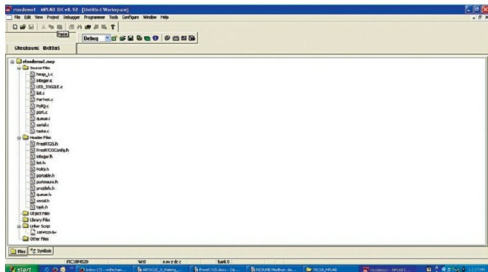


Fig. 1: Untitled Workspace screen

the time by which tasks will execute can be predicted deterministically on the basis of knowledge about the system's hardware and software.

An RTOS divides time between various activities in such a way that all deadlines are met.

An RTOS includes:

1. Support of multiple tasks running concurrently
2. A scheduler to determine which task should run first
3. Ability for the scheduler to preempt a running task
4. Support for inter-task communication

What is FreeRTOS

It is a class of RTOS that is designed to be small and simple enough to run on an MCU; however, its use is not limited to MCU applications. MCUs are used in deeply-embedded applications that normally have a very specific and dedicated job to do. Size constraints and its dedicated end-application nature rarely warrant the use of a full RTOS implementation or even make the use of a full RTOS implementation possible.

FreeRTOS therefore provides core real-time scheduling functionality, inter-task communication, timing and

synchronisation primitives. This means it is more accurately described as a real-time kernel or real-time executive.

Additional functionality such as a command console interface or networking stacks can then be included with add-on components. FreeRTOS has become the *de facto* standard RTOS for MCUs by removing common objections to using free software and, in so doing, provides a truly compelling free software model.

FreeRTOS is a real-time kernel or real-time scheduler on which applications can be built to meet hard real-time requirements. The kernel itself consists of only three or four C files. To make the code readable, easy to port and maintainable, it is written mostly in C, but there are a few assembly functions included, where needed (mostly in architecture-specific scheduler routines).

FreeRTOS architecture

FreeRTOS's codes break down into three main areas: tasks, communication and hardware interfacing.

Tasks. A task is a user-defined C function with a given priority. `tasks.c` and `task.h` do all the heavy lifting for creating, scheduling and maintaining tasks.